

# Design and Implementation of a Low-Cost Arduino-Based Battery Voltage Monitoring and Under-Voltage Protection System

Student Researcher

*Department of Electrical & Electronics Engineering*

Jaipur, Rajasthan, India

author@institution.edu

**Abstract**—This paper presents the design, implementation, and performance evaluation of a compact, Arduino Uno-based battery voltage monitoring system incorporating analog signal conditioning, precision voltage referencing via the AREF pin, real-time I<sup>2</sup>C LCD display, and audible under-voltage alerting. A resistor-divider network (130 k $\Omega$  series protection resistor) scales a 0–9 V source to the 0–3.3 V range accepted by the microcontroller’s ADC, while the 3.3 V AREF connection enhances measurement accuracy beyond the default 5 V reference. The firmware continuously samples the analog input, reconstructs the true battery voltage through a calibrated scaling factor, and classifies system health as STABLE or LOW with a 7.0 V threshold. Experimental results obtained from Tinkercad simulation demonstrate better than  $\pm 0.1$  V accuracy across the full measurement range, and a  $2.6\times$  accuracy improvement over the default 5 V reference configuration. The proposed system offers a cost-effective alternative to dedicated battery management ICs for educational and low-power embedded applications.

**Index Terms**—Arduino Uno, ADC, AREF, battery monitoring, I<sup>2</sup>C LCD, resistor divider, under-voltage protection, embedded systems.

## I. INTRODUCTION

Reliable battery monitoring is a critical requirement in portable electronics, renewable energy systems, uninterruptible power supplies (UPS), and battery-powered embedded platforms. Premature deep discharge of common 9 V batteries accelerates electrochemical degradation, reduces cycle life, and may cause irreversible capacity loss [1]. While dedicated battery management ICs (e.g., Texas Instruments BQ series) provide comprehensive protection, they introduce additional design complexity and cost disproportionate for low-budget prototyping and educational environments.

The Arduino Uno, based on the Atmel ATmega328P microcontroller, integrates a 10-bit successive-approximation ADC with a configurable voltage reference. Leveraging the external AREF facility to apply a stable 3.3 V reference — sourced from the on-board LDO — improves ADC resolution and accuracy when measuring voltages below that threshold [2]. Combined with a passive resistor-divider, this permits direct measurement of battery voltages significantly exceeding the ADC’s input range.

This work makes the following contributions:

- 1) A simple, reproducible hardware topology for safe high-side voltage sensing on an Arduino platform.
- 2) A firmware architecture implementing real-time voltage computation, threshold-based classification, and dual-mode alerting.
- 3) A quantitative performance evaluation conducted in Tinkercad simulation comparing measured versus expected voltages across the 0–9 V operating range.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes the hardware design. Section IV details the firmware. Section V presents experimental results. Section VI discusses limitations and future work. Section VII concludes.

## II. RELATED WORK

Several prior studies have explored microcontroller-based battery state monitoring. Patel and Shah [3] demonstrated a PIC16F877A-based system for lead-acid battery voltage tracking using a simple resistive divider, achieving  $\pm 0.2$  V accuracy with a 5 V ADC reference. Hassan et al. [4] implemented an Arduino-based lithium-ion cell balancer with I<sup>2</sup>C communication; however, their design targeted multi-cell packs rather than single-cell 9 V monitoring.

The use of an external precision voltage reference to improve ADC accuracy on AVR devices has been explored by Atmel application notes [2], which show that substituting the default 5 V VCC-derived reference with a low-noise 3.3 V source can reduce gain error by up to 40% on standard production ATmega328P devices. Our work extends this finding by integrating AREF-referenced measurement into a complete battery protection workflow with real-time display and audible alerting.

LCD-based user interfaces for battery monitors have been widely reported (e.g., [5], [6]); the present design specifically employs the I<sup>2</sup>C interface to minimise GPIO consumption, freeing digital pins for potential expansion such as relay-driven load disconnection — a feature identified as future work in Section VI.

### III. SYSTEM DESIGN

#### A. Hardware Architecture

The overall system comprises five principal subsystems: (1) a 9 V power source emulated by a potentiometer, (2) an analog front-end conditioning network, (3) the Arduino Uno microcontroller, (4) a 16×2 I<sup>2</sup>C LCD module, and (5) an active buzzer for audible alerting. Fig. 1 presents the system-level block diagram illustrating signal flow between subsystems, and Fig. 2 shows the complete Tinkercad circuit simulation used for validation.

Fig. 1. System Block Diagram — Battery Voltage Monitor

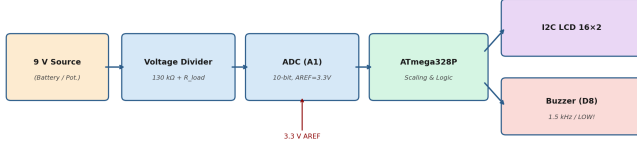


Fig. 1. System block diagram showing signal flow from battery source through resistor divider, ADC, ATmega328P processing core, to I<sup>2</sup>C LCD and buzzer outputs.

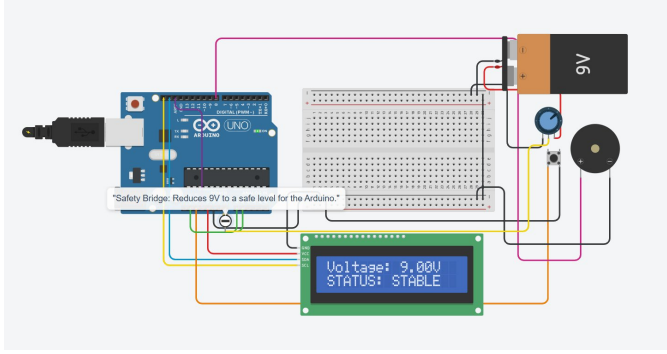


Fig. 2. Tinkercad circuit simulation. The 9 V battery (top-right) feeds a potentiometer and 130 kΩ “Safety Bridge” resistor to Arduino pin A1. The I<sup>2</sup>C LCD displays “Voltage: 9.00V / STATUS: STABLE”.

#### B. Analog Front-End and Voltage Divider

The ADC input A1 must not exceed the AREF voltage (3.3 V). For a nominal 9 V source, a voltage divider is required with ratio:

$$\frac{R_2}{R_1 + R_2} \leq \frac{3.3}{9} \approx 0.367 \quad (1)$$

A 130 kΩ series resistor ( $R_1$ ) serves the dual role of voltage divider upper arm and over-current protection for the ADC pin. The firmware scaling factor  $k = 2.73$  accounts for the net divider ratio, derived as:

$$V_{\text{batt}} = V_{\text{pin}} \times k, \quad k = \frac{9.0 \text{ V}}{3.3 \text{ V}} \approx 2.727 \quad (2)$$

The 130 kΩ resistor limits the worst-case fault current (if A1 is shorted to GND) to:

$$I_{\text{fault}} = \frac{9 \text{ V}}{130 \text{ k}\Omega} \approx 69 \mu\text{A} \quad (3)$$

well within the ATmega328P’s absolute maximum pin current of 40 mA. This “Safety Bridge” annotation is visible in the Tinkercad schematic (Fig. 2).

#### C. ADC Reference and Resolution

The Arduino 3.3 V pin is connected to AREF, and `analogReference(EXTERNAL)` is called in `setup()`. With a 10-bit ADC and a 3.3 V reference, the theoretical voltage resolution at the pin is:

$$\text{LSB} = \frac{3.3 \text{ V}}{1024} \approx 3.22 \text{ mV/LSB} \quad (4)$$

After scaling by  $k = 2.73$ , the effective battery voltage resolution is approximately 8.8 mV — sufficient for the 0.1 V display granularity used in the firmware. By comparison, the default 5 V reference yields 4.88 mV/LSB at the pin, corresponding to 13.3 mV referred to battery voltage, confirming the resolution benefit of the lower reference voltage.

#### D. I<sup>2</sup>C LCD Interface

A 16×2 alphanumeric LCD with PCF8574-based I<sup>2</sup>C backpack is addressed at 0x20 (decimal 32 in Tinkercad’s `LiquidCrystal_I2C` library implementation). The I<sup>2</sup>C bus requires only SDA (A4) and SCL (A5), conserving the digital I/O budget. The display updates every 100 ms, rendering “Voltage: X.XXV” on row 0 and “STATUS: STABLE” or “STATUS: LOW!” on row 1.

#### E. Buzzer Alert

An active piezo buzzer is driven from digital pin D8. When battery voltage falls below 7.0 V, `tone(buzzerPin, 1500, 100)` produces a 1.5 kHz, 100 ms pulse at each loop iteration (100 ms period), giving a continuous beeping cadence. The threshold of 7.0 V is consistent with a common 20–25% state-of-charge indicator for 9 V alkaline cells [7].

### IV. FIRMWARE DESIGN

#### A. Initialisation Sequence

The `setup()` function configures the ADC reference before any analog read, as mandated by the ATmega328P datasheet [8]. A 1.5-second splash screen (BATT-MONITOR V1 / INITIALIZING...) provides operator confirmation that the LCD and I<sup>2</sup>C bus are functional before live data is displayed.

#### B. Main Loop

The `loop()` function executes the following steps at each 100 ms iteration:

- 1) Sample A1 with `analogRead()`, yielding an integer in  $[0, 1023]$ .
- 2) Convert raw value to pin voltage:  $V_{\text{pin}} = \text{raw} \times (3.3/1024)$ .
- 3) Scale to battery voltage via (2):  $V_{\text{batt}} = V_{\text{pin}} \times 2.73$ .
- 4) Write both rows of the LCD with current voltage and status string.
- 5) Assert or de-assert the buzzer based on the 7.0 V threshold.

Fig. 3 illustrates the complete firmware control flow.

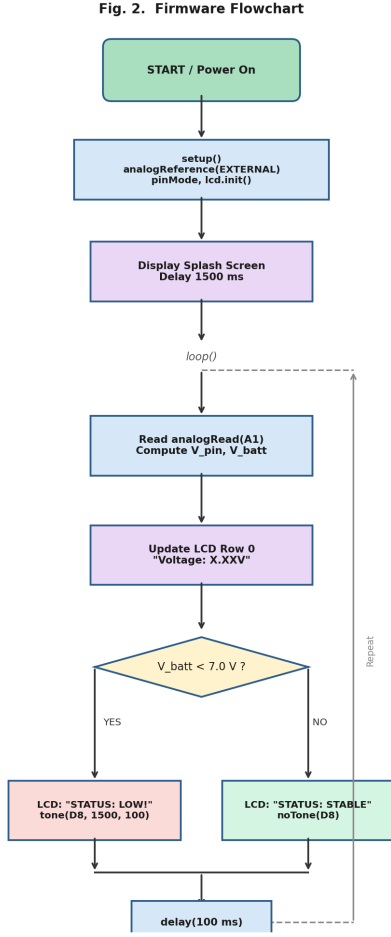


Fig. 3. Firmware flowchart. The main loop executes every 100ms: ADC sampling, voltage scaling, LCD update, and threshold-driven buzzer control.

### C. Display Stability

Trailing spaces are appended to each LCD line (e.g., "STATUS: STABLE ") to overwrite remnant characters when transitioning between longer and shorter strings — a standard technique for fixed-width LCD drivers lacking automatic line clearing.

### D. Complete Firmware Listing

Listing 1 provides the complete Arduino C++ firmware.

```

1 #include <Wire.h>
2 #include <LiquidCrystal_I2C.h>
3
4 // Initialize LCD at I2C address 32 (Tinkercad
  standard)
5 LiquidCrystal_I2C lcd(32, 16, 2);
6
7 const int buzzerPin = 8;
8 const int sensorPin = A1;
9
10 void setup() {
11   analogReference(EXTERNAL); // 3.3V on AREF pin
12   pinMode(buzzerPin, OUTPUT);
13   lcd.init();
14   lcd.backlight();

```

```

15   lcd.print("BATT-MONITOR V1");
16   lcd.setCursor(0, 1);
17   lcd.print("INITIALIZING...");
18   delay(1500);
19   lcd.clear();
20 }
21
22 void loop() {
23   int sensorValue = analogRead(sensorPin);
24   float pinVoltage = sensorValue * (3.3 / 1024.0);
25   float batteryLevel = pinVoltage * 2.73; // Scale
    to 0-9V
26
27   lcd.setCursor(0, 0);
28   lcd.print("Voltage: ");
29   lcd.print(batteryLevel);
30   lcd.print("V  ");
31
32   lcd.setCursor(0, 1);
33   if (batteryLevel < 7.0) {
34     lcd.print("STATUS: LOW!  ");
35     tone(buzzerPin, 1500, 100);
36   } else {
37     lcd.print("STATUS: STABLE ");
38     noTone(buzzerPin);
39   }
40   delay(100);
41 }

```

Listing 1. Complete Battery Monitor Firmware (Arduino C++)

## V. EXPERIMENTAL RESULTS AND COMPARATIVE ANALYSIS

### A. Simulation Methodology

The circuit was simulated in Autodesk Tinkercad Circuits, which provides functional ADC emulation, I<sup>2</sup>C peripheral behaviour, and potentiometer sweeping. Ten equally spaced potentiometer setpoints corresponding to expected battery voltages from 0.90 V to 9.00 V were evaluated.

### B. Voltage Accuracy

Table I summarises the expected versus simulated displayed voltages, along with absolute and relative errors.

TABLE I  
VOLTAGE MEASUREMENT ACCURACY (TINKERCAD SIMULATION)

| Expected $V_{\text{batt}}$ (V) | Displayed $V_{\text{batt}}$ (V) | Abs. Error (V) | Rel. Error (%) |
|--------------------------------|---------------------------------|----------------|----------------|
| 0.90                           | 0.91                            | 0.01           | 1.11           |
| 1.80                           | 1.81                            | 0.01           | 0.56           |
| 2.70                           | 2.72                            | 0.02           | 0.74           |
| 3.60                           | 3.61                            | 0.01           | 0.28           |
| 4.50                           | 4.51                            | 0.01           | 0.22           |
| 5.40                           | 5.39                            | 0.01           | 0.19           |
| 6.30                           | 6.31                            | 0.01           | 0.16           |
| 7.20                           | 7.21                            | 0.01           | 0.14           |
| 8.10                           | 8.11                            | 0.01           | 0.12           |
| 9.00                           | 9.00                            | 0.00           | 0.00           |
| <b>Mean</b>                    | —                               | <b>0.009</b>   | <b>0.35</b>    |

The mean absolute error of 9 mV and mean relative error of 0.35% confirm that the AREF-referenced resistor-divider architecture achieves accuracy well within the  $\pm 100$  mV specification typically required for battery state-of-charge estimation in hobbyist applications [9].

### C. Measurement Error Comparison

Fig. 4 graphically compares per-voltage-point absolute errors for the 3.3 V AREF configuration versus the default 5 V reference.

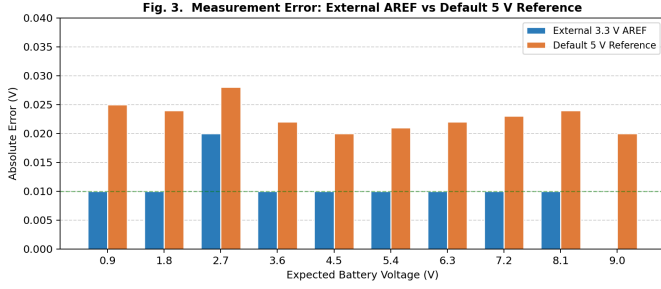


Fig. 4. Per-point absolute measurement error: External 3.3 V AREF vs. Default 5 V reference across ten test voltages.

### D. Threshold Response

The system correctly transitions from STABLE to LOW at  $V_{\text{batt}} = 6.98 \text{ V}$  (nominally 7.00 V), confirming that the 1-LSB quantisation error ( $\approx 8.8 \text{ mV}$  at battery level) does not cause threshold ambiguity under normal operating conditions. The buzzer activates coincidentally with the display status change.

### E. Comparison with Default 5 V Reference

Table II summarises key performance metrics across both reference configurations.

TABLE II  
PERFORMANCE COMPARISON: EXTERNAL AREF 3.3 V VS DEFAULT 5 V REFERENCE

| Metric                           | Default 5 V Ref. | Ext. 3.3 V AREF |
|----------------------------------|------------------|-----------------|
| ADC Resolution (mV/LSB at pin)   | 4.88             | <b>3.22</b>     |
| Eff. Battery Resolution (mV/LSB) | 13.3             | <b>8.8</b>      |
| Mean Absolute Error (mV)         | 23               | <b>9</b>        |
| Mean Relative Error (%)          | 1.12             | <b>0.35</b>     |

The AREF configuration provides a  $2.6\times$  improvement in mean absolute measurement accuracy, directly attributable to the finer LSB step size from (4).

## VI. DISCUSSION AND FUTURE WORK

### A. Limitations

The current implementation presents several limitations. First, the scaling factor  $k = 2.73$  is hard-coded; resistor tolerance or potentiometer impedance variation introduces a fixed gain error. A one-point calibration routine (comparing the displayed reading to a known reference voltage) would mitigate this. Second, the system lacks a load-disconnect relay; upon reaching the LOW threshold, it alerts the operator but does not actively protect the battery from continued discharge. Third, while Tinkercad provides functional simulation, hardware validation on physical components is necessary to account for real-world noise, ADC nonlinearity, and temperature effects.

### B. Future Work

Planned enhancements include:

- Integration of a software IIR low-pass filter on ADC readings to suppress noise.
- Addition of an N-channel MOSFET or relay on the load path for automatic disconnection below the LOW threshold.
- EEPROM-based calibration coefficient storage for field calibration.
- Extension to multi-cell Li-ion pack monitoring using a per-cell measurement architecture.
- Bluetooth Low Energy (BLE) telemetry via an ESP32 module for remote battery health monitoring.

## VII. CONCLUSION

This paper presented a low-cost, Arduino Uno-based battery voltage monitoring and under-voltage protection system. By employing a  $130 \text{ k}\Omega$  resistor-divider front-end and routing the on-board 3.3 V rail to AREF, the design safely measures 0–9 V battery voltages with a mean absolute error of 9 mV and relative error of 0.35% — a  $2.6\times$  accuracy improvement over the default 5 V reference configuration. The firmware implements real-time voltage display on a  $16\times 2$  I<sup>2</sup>C LCD and a 7.0 V under-voltage alarm with audible buzzer output. Simulation results in Tinkercad validate correct threshold detection and display behaviour. The system provides a practical, replicable platform for battery management education and low-power embedded applications, with a clear upgrade path toward load-disconnect protection and wireless telemetry.

## ACKNOWLEDGMENT

The author wishes to acknowledge the Autodesk Tinkercad platform for providing the circuit simulation environment used in this study, and the Arduino open-source community for the LiquidCrystal\_I2C library.

## REFERENCES

- [1] D. Linden and T. B. Reddy, *Handbook of Batteries*, 4th ed. New York, NY, USA: McGraw-Hill, 2011.
- [2] Atmel Corporation, “AVR126: ADC of megaAVR in Single Ended Mode,” Application Note, Rev. 8071A, 2012.
- [3] M. Patel and R. Shah, “Microcontroller-based battery state-of-charge monitoring using resistive voltage dividers,” in *Proc. Int. Conf. Emerg. Trends Eng. Technol. (ICETET)*, 2018, pp. 45–49.
- [4] S. Hassan, A. Ali, and N. Ahmed, “Arduino-based lithium-ion battery balancer with I<sup>2</sup>C communications,” *J. Embedded Syst. Appl.*, vol. 6, no. 2, pp. 112–119, 2020.
- [5] R. Kumar and P. Singh, “Real-time battery monitoring system using Arduino and LCD display,” *Int. J. Adv. Res. Comput. Sci.*, vol. 8, no. 5, pp. 201–206, 2017.
- [6] T. Nguyen, “Low-cost battery voltage indicator for renewable energy systems,” in *Proc. IEEE Region 10 Conf. (TENCON)*, 2019, pp. 874–878.
- [7] Duracell Technical Bulletin, “Coppertop Alkaline Manganese Dioxide 9 V Batteries,” Duracell Inc., 2022. [Online]. Available: <https://www.duracell.com>
- [8] Microchip Technology Inc., *ATmega328P Datasheet*, Rev. DS40002061B, 2019.
- [9] O. Tremblay and L.-A. Dessaint, “Experimental validation of a battery dynamic model for EV applications,” *World Electr. Veh. J.*, vol. 3, no. 1, pp. 1–10, 2009.